MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automatics

Mihail Pancenco FAF-243

Report

Laboratory Work No. 3

Cryptography

Playfair Cipher

Romanian alphabet

Task 3.1 – Variant 19

Supervisors:

Poștaru Andrei, *Doctor*

Maia Zaica, *University assistant*

DISA, FCIM, UTM

Chișinău – 2026

Table of Contents

1 Task and short theory **3**

1.1 Task 3.1 and the Romanian alphabet 3

1.2 Encryption and decryption rules 3

2 Implementation **4**

2.1 Program structure, normalization and validation 4

2.2 Building the keyword matrix 5

2.3 Preparing pairs and inserting fillers 6

2.4 Transforming pairs and reversing the cipher 7

3 Results and verification **8**

3.1 Manual check of all three rules 8

3.2 Repeated letters and decryption 9

3.3 Romanian text and a permuted keyword matrix 10

3.4 Validation and input recovery 10

4 Conclusions **11**

4.1 Verification coverage and source code 11

Bibliography **11**

1 Task and short theory

1.1 Task 3.1 and the Romanian alphabet

The objective of Laboratory Work No. 3 is to study classical ciphers that use more than a fixed substitution of individual letters. My number in the group list is 19. Since it is odd, the selected task is 3.1: implement the Playfair cipher for Romanian messages, as specified in the assignment [1]. The program must accept encryption and decryption, require a key of at least seven letters, reject invalid input clearly, and leave the final restoration of word spaces to the user.

Playfair encrypts pairs of letters using a matrix built from a keyword. The Romanian alphabet has 31 letters. In this implementation, J is treated as I, so the matrix has 30 cells arranged in **six rows and five columns**. The letter Î remains distinct from I. Uppercase and lowercase Romanian letters are accepted; the cedilla variants of Ș and Ț are normalized.

The keyword's first occurrences are written into the matrix first. All unused letters are then appended in Romanian alphabetical order. Messages are converted to uppercase, ordinary spaces are removed, and J is replaced by I. Identical letters within a pair are separated with X; Q is used when the repeated letter is X. An unmatched final letter receives the same choice of filler. Decryption keeps these fillers for manual interpretation.

1.2 Encryption and decryption rules

Let $A[r,c]$ be the matrix, using zero-based coordinates $r \in \{0,\ldots,5\}$ and $c \in \{0,\ldots,4\}$. Let $d = 1$ for encryption and $d = -1$ for decryption. For a pair at positions (r_1,c_1) and (r_2,c_2):

- **Same row:** keep each row and replace each column by $(c+d+5) \bmod 5$.
- **Same column:** keep each column and replace each row by $(r+d+6) \bmod 6$.
- **Rectangle:** use the opposite corners (r_1,c_2) and (r_2,c_1). This rule is identical for encryption and decryption.

Thus encryption moves right or down, while decryption moves left or up. The modulo operations implement wraparound. Adding the relevant dimension keeps the remainder nonnegative during decryption.

Playfair substitutes letter pairs, which obscures simple single-letter frequency relationships. It is nevertheless a classical educational cipher, not a suitable method for protecting modern confidential communication.

2 Implementation

2.1 Program structure, normalization and validation

The C# implementation contains two classes. `Program` manages the menu, reads input, catches validation errors and repeats the corresponding prompt. `PlayfairCipher` builds the matrix, prepares letter pairs, encrypts and decrypts. The project targets .NET 10. The excerpts below are taken directly from the current source [2]; empty lines are compressed for readability.

```
private static string Normalize(string text)
{
    ArgumentNullException.ThrowIfNull(text);
    text = text.Replace('ş', 'ș')
        .Replace('Ş', 'Ș')
        .Replace('ţ', 'ț')
        .Replace('Ţ', 'Ț')
        .ToUpperInvariant()
        .Replace('J', 'I')
        .Replace(" ", "");
    foreach (char letter in text)
    {
        if (!Alphabet.Contains(letter))
        {
            throw new ArgumentException(
                $"Character '{letter}' is not allowed. Use Romanian letters "
                    + "A-Z, Ă, Â, Î, Ș, Ț (uppercase or lowercase). "
                    + "Ordinary spaces are allowed in messages."
            );
        }
    }
    return text;
}
```

Comment. `Normalize` replaces cedilla variants, converts text to uppercase, merges J into I and removes ordinary spaces. It then checks every remaining character against the 30 matrix symbols. Digits, punctuation, tabs and other unsupported characters are rejected. The error identifies the rejected character and states the accepted letters.

```
    if (keyword.Contains(' '))
    {
        throw new ArgumentException("The keyword must contain letters only, without spaces.");
    }
    keyword = Normalize(keyword);
    if (keyword.Length < 7)
    {
        throw new ArgumentException("The keyword must contain at least 7 letters before removing
duplicates.");
    }
```

Comment. The constructor forbids spaces in the keyword, then normalizes it and enforces a minimum length of seven letters. This check occurs *before* duplicate removal; repeated letters are allowed in a valid keyword. Empty messages are rejected by `Encrypt`, and `Decrypt` requires a nonempty, even-length ciphertext after normalization.

2.2 Building the keyword matrix

```
    private const string Alphabet = "AĂÂBCDEFGHIÎKLMNOPQRSŞTŢUVWXYZ";
    private const int Rows = 6;
    private const int Columns = 5;
    private readonly char[,] matrix = new char[Rows, Columns];
```

Comment. The matrix alphabet contains 30 symbols because J uses the I cell. The dimensions are six rows and five columns.

```
    private void BuildMatrix(string keyword)
    {
        string letters = "";
        foreach (char letter in keyword)
        {
            if (!letters.Contains(letter))
            {
                letters += letter;
            }
        }
        Console.WriteLine("Unique key: " + letters);
        foreach (char letter in Alphabet)
        {
            if (!letters.Contains(letter))
            {
                letters += letter;
            }
        }
        int index = 0;
        for (int row = 0; row < Rows; row++)
        {
            for (int column = 0; column < Columns; column++)
            {
                matrix[row, column] = letters[index];
                index++;
            }
        }
    }
```

Comment. The first loop retains each keyword letter once, in order of first appearance. The second loop appends unused alphabet letters. The nested loops fill the matrix from left to right, row by row. This produces exactly one occurrence of every matrix symbol.

For example, the key `criptografie` becomes `CRIPTOGRAFIE`; its distinct letters are `CRIPTOGAFE`. The resulting matrix is

C	R	I/J	P	T
O	G	A	F	E
Ă	Â	B	D	H
Î	K	L	M	N
Q	S	Ș	Ț	U
V	W	X	Y	Z

`FindPosition` locates a letter by scanning the rows and columns. `PrintMatrix` displays the same cells, labelling the I cell as I/J.

2.3 Preparing pairs and inserting fillers

`PreparePairs` scans the normalized message using an index. The first letter of the next pair is `message[index]`. The filler is X, except when that letter is itself X, in which case the filler is Q. The following branches decide the second letter and how far to advance:

```
if (index + 1 == message.Length)
{
    second = filler;
    Console.WriteLine($"Add {filler} after the final {first}.");
    index++;
}
else if (first == message[index + 1])
{
    second = filler;
    Console.WriteLine($"Insert {filler} between repeated {first} letters.");
    index++; // The second original letter starts the next pair.
}
else
{
    second = message[index + 1];
    index += 2;
}
pairs.Add(first.ToString() + second);
```

Comment. If only one letter remains, the method appends a filler and consumes one original letter. If the next two letters are identical, it forms a pair with a filler and also advances by only one: the second original letter must be reconsidered. Otherwise, it consumes both letters. Each constructed pair is added to a `List<string>`.

Table 2.1 - Pair preparation for BALLOON

Index	Next letters	Pair produced	Next index
0	BA	BA	2
2	LL	LX	3
3	LO	LO	5
5	ON	ON	7

The prepared text is `BALXLOON`, or `BA LX LO ON`. Only repeated letters inside the current pair are separated. For example, `ABBA` already splits into `AB BA` and needs no filler. For `XX`, the method produces `XQ XQ`; for a final X, it produces `XQ`. This prevents an identical-letter filler pair.

2.4 Transforming pairs and reversing the cipher

```
private string TransformPair(string pair, int direction)
{
    FindPosition(pair[0], out int firstRow, out int firstColumn);
    FindPosition(pair[1], out int secondRow, out int secondColumn);
    string rule;
    if (firstRow == secondRow)
    {
        firstColumn = (firstColumn + direction + Columns) % Columns;
        secondColumn = (secondColumn + direction + Columns) % Columns;
        rule = "same row";
    }
    else if (firstColumn == secondColumn)
    {
        firstRow = (firstRow + direction + Rows) % Rows;
        secondRow = (secondRow + direction + Rows) % Rows;
        rule = "same column";
    }
    else
    {
        int savedColumn = firstColumn;
        firstColumn = secondColumn;
        secondColumn = savedColumn;
        rule = "rectangle";
    }
    string result = matrix[firstRow, firstColumn].ToString() + matrix[secondRow, secondColumn];
    Console.WriteLine($"{pair} -> {result}  {rule}");
    return result;
}
```

Comment. `FindPosition` supplies the coordinates of the two input letters. The three branches implement the same-row, same-column and rectangle rules. `Encrypt` passes direction 1; `Decrypt` passes direction -1. The result letters are read from the matrix, and one short console line shows the input pair, output pair and rule used.

During encryption, the message is normalized and sent through `PreparePairs`. During decryption, the ciphertext is normalized and split directly into two-letter substrings; no fillers are inserted. Both methods concatenate the transformed pairs and return a string.

The console prints the normalized key, distinct keyword letters, matrix, normalized message and prepared pairs before the transformations. Filler insertions are reported where they occur. The final output is either the ciphertext or the recovered prepared plaintext. Word spaces, capitalization, the original I/J choice and ambiguous filler letters are not reconstructed automatically.

3 Results and verification

3.1 Manual check of all three rules

The project builds successfully with zero compiler warnings and errors. The console transcripts in this chapter were captured from the running, unchanged program. For a matrix that is especially easy to verify by hand, I used the seven-letter key `AĂÂBCDE`. Its matrix follows the base alphabet order:

	1	2	3	4	5
1	A	Ă	Â	B	C
2	D	E	F	G	H
3	I/J	Î	K	L	M
4	N	O	P	Q	R
5	S	Ș	T	Ț	U
6	V	W	X	Y	Z

The coordinates in Table 3.1 start at one for convenient manual checking. The code uses zero-based array indices.

Table 3.1 - Manual encryption and inverse checks

Input	Input coordinates	Rule	Cipher pair	Decrypts to
AC	(1,1), (1,5)	Same row	ĂA	AC
AV	(1,1), (6,1)	Same column	DA	AV
AȚ	(1,1), (5,4)	Rectangle	BS	AȚ

For AC, C wraps from column 5 to column 1. For AV, V wraps from row 6 to row 1. For AȚ, the opposite rectangle corners contain B and S.

The diagnostic input `AC AV AȚ` exercises all three rules in one run:

```
Normalized: ACAVAȚ
Pairs: AC AV AȚ
AC -> ĂA  same row
AV -> DA  same column
AȚ -> BS  rectangle

Ciphertext: ĂADABS
```

Decrypting `ĂA DA BS` with the same key gives `ACAVAȚ`. This confirms the inverse left/up shifts and the unchanged rectangle rule.

3.2 Repeated letters and decryption

For key AĂÂBCDE and input bal loon, the program removes the input space, converts the text to uppercase and inserts X between the repeated L letters:

```
Normalized: BALLOON
Insert X between repeated L letters.
Pairs: BA LX LO ON
BA -> CĂ  same row
LX -> KY  rectangle
LO -> ÎQ  rectangle
ON -> PO  same row

Ciphertext: CĂKYÎQPO
```

With the same key, decrypting CĂ KY ÎQ PO returns the prepared plaintext rather than deleting the inserted X:

```
Normalized: CĂKYÎQPO
Pairs: CĂ KY ÎQ PO
CĂ -> BA  same row
KY -> LX  rectangle
ÎQ -> LO  rectangle
PO -> ON  same row

Decrypted text: BALXLOON
Remove inserted fillers and restore spaces manually; I/J share a cell.
```

The result BALXLOON agrees with the prepared encryption input. The user can identify the inserted X and restore the intended word spacing. Removing every X automatically would also remove genuine plaintext letters.

For the boundary case XX, the program prints Pairs: XQ XQ and produces YPYP. For a single input A, it appends X, prints Pairs: AX and produces ÂV. These results confirm both filler selection and final-letter padding.

3.3 Romanian text and a permuted keyword matrix

I also used the keyword `criptografie` and the Romanian input `ştiinţă şi ţară`. The cedilla variants are accepted and normalized to comma-below letters; the resulting matrix is the one shown in Section 2.2.

```
Normalized: ȘTIINȚĂȘIȚARĂ
Insert X between repeated I letters.
Pairs: ȘT IX IN ȚĂ ȘI ȚA RĂ
ȘT -> UI  rectangle
IX -> AI  same column
IN -> TL  rectangle
ȚĂ -> QD  rectangle
ȘI -> XA  same column
ȚA -> ȘF  rectangle
RĂ -> CÂ  rectangle

Ciphertext: UIAITLQDXAȘFCÂ
```

The ciphertext is `UIAITLQDXAȘFCÂ`. Decrypting it with the same key returns `ȘTIXINȚĂȘIȚARĂ`: the same normalized message with the inserted X retained. This run checks Romanian diacritics, lowercase input, spaces, duplicate keyword letters and a keyword-dependent alphabet order.

3.4 Validation and input recovery

The program rejects invalid input and repeats the corresponding prompt. Table 3.2 summarizes captured validation runs; the full console transcripts are saved in `evidence/`.

Table 3.2 - Input validation cases and observed responses

Field	Input	Response
Keyword	`secret`	Minimum length is seven letters before removing duplicates.
Keyword	`cripto1`	Character 1 is rejected; the permitted Romanian letters are stated.
Keyword	`cripto grafie`	Spaces in the keyword are rejected.
Message	Empty input	At least one Romanian letter is required.
Message	`salut!`	Character ! is rejected; the permitted Romanian letters are stated.
Message	`salut2`	Character 2 is rejected; the permitted Romanian letters are stated.
Ciphertext	`ABC`	A nonzero, even number of letters is required.

After invalid keys, supplying `AĂÂBCDE` continues the operation. After invalid message characters, `Salut` is accepted. After the odd-length ciphertext `ABC`, supplying `ĂADABS` successfully decrypts to `ACAVAȚ`. Thus an error does not terminate the program.

4 Conclusions

In this laboratory work, I implemented the Romanian Playfair cipher using a keyword matrix with six rows and five columns. Merging J into I made room for the 31-letter Romanian alphabet while keeping Î as a separate letter. The program handles uppercase and lowercase input, Romanian cedilla variants and ordinary message spaces.

I verified the three transformation rules and their inverses: horizontal shifts, vertical shifts and rectangle substitution. The examples also confirmed the importance of preparing pairs correctly. When two current letters are identical, only the first original letter is consumed; the second must be reconsidered. X is the usual filler, with Q used when an X would otherwise create an identical pair.

The console output makes the process easy to check: it shows the matrix, prepared pairs and one short explanation for each transformation. Decryption correctly recovers the prepared plaintext. Restoring word spaces, distinguishing an original J from I and resolving filler letters remain manual tasks because this information is not preserved by the cipher. The work reinforced my understanding of pair substitution, modular wraparound and input validation through a small, readable implementation.

4.1 Verification coverage and source code

For each of two keyed matrices, I checked all $30 \times 29 = 870$ ordered pairs of distinct letters against an independent coordinate calculation, then decrypted the results. Both sets passed: 1,740 pair cases in total. The checks also covered repeated letters, alternate fillers, odd-length plaintext, Romanian normalization, the minimum key length, invalid input and decryption back to the prepared plaintext. The verification run passed 3,537 assertions. The source files were not modified while preparing this report.

Source files: `Program.cs` and `PlayfairCipher.cs`, in `lab_03/code/playfair/`. The project repository is

`https://github.com/MiHCraft/Cryptography_UTM`

Bibliography

[1] Technical University of Moldova. *Lucrare de laborator nr. 3: Cifruri polialfabetice*. Course assignment, Section 3.2 (Playfair), pp. 3–6, and Task 3.1, p. 7.

[2] Pancenco, Mihail. *Romanian Playfair Cipher*. C# implementation, `lab_03/code/playfair/Program.cs` and `lab_03/code/playfair/PlayfairCipher.cs`. Project repository: `https://github.com/MiHCraft/Cryptography_UTM`.